

Relative path: run_reasoning.py

Filename: run_reasoning.py Extension: .py

"""

OASIS v2 – run_reasoning.py

Runs Layer 4 over every risk assessment. Run AFTER run_pipeline.py.

Usage:

python3 run_reasoning.py

Optional: export ANTHROPIC_API_KEY=sk-... first for real LLM narratives.

Without it, uses the offline template fallback automatically.

"""

import json

from pathlib import Path

from reasoning.llm_layer import generate_incident_narrative

RESULTS_DIR = Path(__file__).resolve().parent / "results"

def main():

 risk_assessments = json.load(open(RESULTS_DIR / "risk_assessments.json"))

 print(f"Generating incident narratives for {len(risk_assessments)} risk assessments...")

 narratives = []

 n_failed = 0

 for risk in risk_assessments:

 narrative = generate_incident_narrative(risk)

 if not narrative["_groundedness_check"]["passed"]:

 n_failed += 1

 narratives.append(narrative)

 with open(RESULTS_DIR / "incident_narratives.json", "w") as f:

 json.dump(narratives, f, indent=2)

 metrics_path = RESULTS_DIR / "metrics.json"

 if metrics_path.exists():

 metrics = json.load(open(metrics_path))

 else:

 metrics = {}

 grounded_total = len(narratives)

 grounded_passed = grounded_total - n_failed

 metrics["grounded_narratives"] = {

 "passed": grounded_passed,

 "total": grounded_total,

 "percent": round((grounded_passed / grounded_total) * 100, 1) if grounded_total else 0.0,

 }

 with open(metrics_path, "w") as f:

 json.dump(metrics, f, indent=2)

 print(f"Wrote {len(narratives)} narratives -> {RESULTS_DIR / 'incident_narratives.json'}")

 print(f"Groundedness: {len(narratives) - n_failed}/{len(narratives)} passed")

if __name__ == "__main__":

 main()